

README

Auto Generator

AI-Powered Documentation for Every Codebase

Project Report · Version 1.0 · May 2025

Full-Stack Web Application | Node.js · React · Claude AI

Layers	Components	Free APIs	Target Users
5	12+	100 %	Developers

1. Executive Summary

README Auto Generator is an end-to-end, AI-powered web application that eliminates one of the most tedious tasks in software development: writing documentation. A developer pastes a GitHub repository URL (or uploads a zip), and within seconds receives a fully structured, professional README.md — inferred directly from the source code, dependency manifests, folder layout, and existing comments.

The system is architected as a decoupled, three-tier application: a React single-page frontend, a Node.js REST/streaming backend, and an AI generation layer powered by the Anthropic Claude API. Every external service used in the recommended stack is available under a free tier, making this project fully executable at zero cost for development and early production traffic.

Value Proposition	Detail
Zero manual writing	AI generates all README sections from code context
Stack-agnostic	Works with any language / framework combination
Streaming output	Users see the README built section-by-section in real time
Exportable	One-click copy or push to GitHub
100 % free APIs	All third-party services offer generous free tiers
Extensible	GitHub Action hook for auto-regeneration on every push

MOTIVATION

2. Problem Statement

Despite being the first touchpoint for any person exploring a repository, README files are consistently the last thing developers write — and the first thing they neglect to update. The consequences are significant:

- Open-source projects lose potential contributors because setup instructions are missing or outdated.
- Internal tools become silos when onboarding docs lag behind the codebase.
- Writing a quality README from scratch takes 30–90 minutes of unproductive context-switching.
- Existing template generators produce generic skeletons that still require manual editing.

README Auto Generator solves this by treating documentation as a **derived artefact** — automatically inferred from the code itself, not written from scratch by the developer.

ARCHITECTURE

3. System Architecture

The application follows a classic three-tier architecture augmented with an AI generation layer. All tiers communicate over HTTPS/HTTP2 with JSON and Server-Sent Events (SSE) for streaming.

Layer	Technology	Responsibility
Presentation	React 18 + Vite + Tailwind CSS	User interface, live preview, export controls
API Gateway	Node.js 20 + Express 5	Request routing, auth, rate limiting, SSE streaming
Analysis	Node.js workers (in-process)	Repo fetch, code parse, metadata extract, prompt build
AI Generation	Anthropic Claude API (claude-sonnet-4.1-2025-05-14)	Code context-aware README section generation
Persistence	MongoDB Atlas Free / SQLite (dev)	Generation history, user sessions, cached results

3.1 Data Flow Pipeline

The end-to-end pipeline executes five sequential stages:

Stage	Input	Output
① Input	GitHub URL / zip upload	Raw repository files on server disk / memory
② Fetch & Unpack	Repo source	Flat file tree with full paths
③ Analyze	File tree	Structured JSON: stack, deps, scripts, entry-points, docstrings
④ Prompt Build	Structured JSON	Templated prompt with injected code context
⑤ Generate	Prompt → Claude API	Streaming Markdown README sections
⑥ Deliver	SSE stream	Live preview in browser / downloadable README.md

[CODEBASE](#)

4. Project Structure

The repository is organised as a monorepo with three top-level workspaces: **client**, **server**, and **shared**. A root-level *docker-compose.yml* orchestrates all services for local development.

```
readme-auto-generator/  
├── client/                # React frontend (Vite)  
│   ├── public/  
│   └── src/  
│       ├── components/  
│       │   ├── InputPanel/  # URL input, file upload, settings  
│       │   ├── PreviewPane/ # Live streaming Markdown preview  
│       │   └── Toolbar/     # Copy, export, GitHub push controls
```

- ■ ■ ■ HistoryDrawer/ # Past generations list
- ■ ■ ■ hooks/ # useSSE, useGenerate, useAuth
- ■ ■ ■ services/ # API client, GitHub OAuth helper
- ■ ■ ■ stores/ # Zustand state slices
- ■ ■ ■ utils/ # Markdown helpers, file converters

- ■ ■ ■ server/ # Node.js + Express backend

- ■ ■ ■ routes/

- ■ ■ ■ generate.js # POST /api/generate (SSE)

- ■ ■ ■ export.js # POST /api/export

- ■ ■ ■ history.js # GET /api/history

- ■ ■ ■ services/

- ■ ■ ■ repoFetcher.js # GitHub API / zip extractor

- ■ ■ ■ codeAnalyzer.js # AST + regex parser

- ■ ■ ■ metadataExtractor.js # package.json / pom.xml / etc.

- ■ ■ ■ promptBuilder.js # Assembles Claude prompt

- ■ ■ ■ claudeService.js # Streams Claude API responses

- ■ ■ ■ middleware/

- ■ ■ ■ auth.js # JWT / GitHub OAuth

- ■ ■ ■ rateLimiter.js # express-rate-limit

- ■ ■ ■ models/ # Mongoose / Sequelize models

- ■ ■ ■ config/ # env validation (zod)

- ■ ■ ■ index.js # Server entry point

- ■ ■ ■ shared/ # Types & constants (TS)

- ■ ■ ■ types.ts

- ■ ■ ■ .github/

- ■ ■ ■ workflows/

- ■ ■ ■ ci.yml # Lint, test, build on PR

- ■ ■ ■ readme-gen.yml # Auto-generate README on push

- ■ ■ ■ docker-compose.yml # Local dev orchestration

- ■ ■ ■ Dockerfile.client

- ■ ■ ■ Dockerfile.server

- ■ ■ ■ README.md # (Generated by this tool!)

5. Key Modules Explained

repoFetcher.js

Accepts a GitHub repo URL or an uploaded zip archive. For public repos it calls the GitHub REST API (unauthenticated, free) to download the repository as a tarball. For private repos it exchanges a GitHub OAuth token. Zip uploads are unpacked in a temporary in-memory stream. The output is a flat map of { filePath → fileContent } limited to the first 200 non-binary files.

codeAnalyzer.js

Performs lightweight static analysis without executing any code. It identifies: (a) entry-point files via heuristics (main.py, index.js, App.jsx, etc.), (b) technology keywords via regex over file extensions and import statements, (c) function / class names and JSDoc / docstring blocks for the top-level files, and (d) the full directory tree truncated to depth 3.

metadataExtractor.js

Reads well-known manifest files and extracts structured metadata. Supported formats include package.json (Node.js), requirements.txt / pyproject.toml (Python), pom.xml / build.gradle (Java/Kotlin), Gemfile (Ruby), go.mod (Go), Cargo.toml (Rust), Dockerfile, docker-compose.yml, and .github/workflows/*.yml. Outputs a normalised object: { name, description, version, scripts, dependencies, devDependencies, license, author }.

promptBuilder.js

Assembles the final prompt that is sent to Claude. The prompt is a structured template with clearly labelled XML-like sections — , , , — followed by a precise instruction to generate each README section (Title, Description, Tech Stack, Installation, Usage, Folder Structure, API Reference, License, Contributing) with realistic, inferred content rather than placeholder text.

claudeService.js

Calls the Anthropic Messages API with stream=true. It pipes the SSE chunks received from the API directly to the client's SSE connection, so the React preview pane updates token-by-token. On completion it persists the full Markdown output to the database and returns the generation ID.

6. REST API Design

The backend exposes a minimal, resource-oriented REST API. All endpoints are prefixed with **/api/v1**. Streaming responses use the **text/event-stream** content type (SSE).

Method	Endpoint	Auth	Description
POST	/generate	Optional	Analyse repo & stream README via SSE
GET	/generate/:id	Optional	Fetch a previously generated README
POST	/export	Required	Push README.md to a GitHub repo
GET	/history	Required	List user's past generations
DELETE	/history/:id	Required	Delete a saved generation
GET	/auth/github	—	Initiate GitHub OAuth flow
GET	/auth/github/callback	—	OAuth callback, issues JWT
GET	/health	—	Liveness probe

POST /generate — Request Body

Field	Type	Required	Description
repoUrl	string	Yes*	GitHub repository URL (public or private)
branch	string	No	Branch name (default: main / master)
sections	array	No	Subset of README sections to generate
tone	string	No	casual professional technical
includeTree	boolean	No	Include directory tree section (default: true)

* Either repoUrl or a multipart zip file upload is required.

7. Dependencies & Free APIs

Every library and external service listed below is open-source or available on a free tier sufficient for development and early production use. No paid plan is required to build and run this project.

7.1 Frontend (client/)

Package	Version	Purpose
---------	---------	---------

react	^18	UI component framework
react-dom	^18	DOM rendering
vite	^5	Dev server & bundler
@vitejs/plugin-react	^4	React fast-refresh for Vite
react-router-dom	^6	Client-side routing
zustand	^4	Lightweight global state
tailwindcss	^3	Utility-first CSS
react-markdown	^9	Render Markdown preview
remark-gfm	^4	GitHub-Flavoured Markdown plugin
axios	^1	HTTP client
lucide-react	^0.38	Icon library (MIT)
clsx	^2	Conditional className utility

7.2 Backend (server/)

Package	Version	Purpose
express	^5	HTTP framework
@anthropic-ai/sdk	^0.27	Official Claude API client
mongoose	^8	MongoDB ODM (optional, free Atlas tier)
better-sqlite3	^9	SQLite for local / serverless dev
octokit	^4	GitHub REST & GraphQL client
passport	^0.7	OAuth middleware
passport-github2	^0.1	GitHub OAuth 2.0 strategy
jsonwebtoken	^9	JWT generation & verification
express-rate-limit	^7	Rate limiting middleware
adm-zip	^0.5	In-memory zip extraction
tar-stream	^3	Streaming tarball extraction
zod	^3	Runtime schema validation
pino	^9	Structured JSON logging
dotenv	^16	Environment variable loader
cors	^2	CORS headers middleware

helmet	^7	HTTP security headers
jest	^29	Unit & integration testing
supertest	^7	HTTP integration test helper

7.3 External Free APIs & Services

Service	Free Tier Limit	Usage
Anthropic Claude API	Free trial credits	README generation (claude-sonnet-4-20250514)
GitHub REST API	60 req/hr (unauth)	Repo fetch, tarball download
GitHub OAuth Apps	Unlimited (free)	User authentication
GitHub API (authed)	5,000 req/hr	Export README.md to repo
MongoDB Atlas	512 MB free cluster	Persistent storage (optional)
Render.com	750 hr/month free web svc	Backend hosting (optional)
Vercel	Unlimited hobby deploys	Frontend hosting (optional)

OUTPUT

8. Generated README Sections

The AI generation layer produces a Markdown document divided into clearly structured sections. The table below maps each section to the code signals used to infer its content.

README Section	Inferred From
Project Title	Repository name, package.json name field
Description / About	package.json description, top-level docstrings, entry-file comments
Tech Stack / Built With	Dependency files, file extensions, import statements
Prerequisites	Node/Python/Java version in manifest, .nvmrc, .python-version
Installation	Package manager lockfile (npm/yarn/pnpm/pip), install scripts
Configuration	.env.example keys, config schema (zod/joi)
Usage / How to Run	start / dev scripts, CLI argument parsing, main function comments
Folder Structure	Directory tree (depth 3, auto-generated)
API Reference	Express router definitions, OpenAPI comments, JSDoc
Running Tests	test scripts, jest / pytest config
Deployment	Dockerfile, docker-compose.yml, GitHub Actions workflow

Contributing	CONTRIBUTING.md (if present) or generic best-practices
License	LICENSE file type or license field in manifest

BONUS

9. GitHub Action — Auto README

As a bonus feature, the project ships a reusable GitHub Action (`.github/workflows/readme-gen.yml`) that automatically calls the deployed API on every push to `main` and commits the regenerated `README.md` back to the repository. This keeps documentation perpetually in sync with the codebase.

Step	Action
Trigger	push to main branch (configurable)
Checkout	actions/checkout@v4
Call API	curl POST /api/generate with GITHUB_TOKEN
Write file	Save streamed Markdown to README.md
Commit & push	stefanzweifel/git-auto-commit-action@v5 (free)

NFR

10. Non-Functional Requirements

Attribute	Target	Implementation
Performance	< 3 s to first streamed token	SSE streaming, prompt caching
Scalability	Stateless API workers	Horizontal scaling via Docker / Render
Security	No secrets in logs / client	Env vars, helmet, JWT HttpOnly cookie
Reliability	Graceful API error handling	Retry with exponential back-off (3 ×)
Privacy	Repo content not persisted	Only generated Markdown is stored
Accessibility	WCAG 2.1 AA	Semantic HTML, aria labels, keyboard nav
Observability	Structured logs	pino → stdout (Cloud Run / Render logs)

ROADMAP

11. Development Roadmap

Phase 1 — Core MVP (Weeks 1–3)

- ✓ Project scaffold (monorepo, linting, CI)
- ✓ GitHub public repo fetch via Octokit
- ✓ Code analyzer & metadata extractor
- ✓ Basic prompt builder + Claude API call (non-streaming)
- ✓ React UI: URL input + static Markdown preview
- ✓ Copy-to-clipboard export

Phase 2 — Streaming & Auth (Weeks 4–5)

- ✓ Switch Claude call to streaming (SSE)
- ✓ React live-preview with token-by-token update
- ✓ GitHub OAuth login
- ✓ MongoDB Atlas persistence (history)
- ✓ Rate limiting + error boundary UI

Phase 3 — Advanced Features (Weeks 6–7)

- ✓ ZIP file upload support
- ✓ Section selector (choose which sections to generate)
- ✓ Tone selector (casual / professional / technical)
- ✓ Push README to GitHub repo (Export button)
- ✓ GitHub Action workflow file

Phase 4 — Polish & Deploy (Week 8)

- ✓ Docker Compose for local dev
- ✓ Vercel deploy (frontend) + Render deploy (backend)
- ✓ End-to-end tests (Playwright)
- ✓ Performance profiling & prompt-cache optimisation
- ✓ Documentation & demo video

12. Environment Variables

All secrets and configuration are passed through environment variables. A **.env.example** file is committed to the repository; the actual **.env** file is git-ignored.

Variable	Required	Description
ANTHROPIC_API_KEY	Yes	Claude API key from console.anthropic.com
GITHUB_CLIENT_ID	Yes	GitHub OAuth App client ID
GITHUB_CLIENT_SECRET	Yes	GitHub OAuth App client secret
GITHUB_TOKEN	No	Personal access token (fallback for private repos)
JWT_SECRET	Yes	Random string for signing JWTs (min 32 chars)
MONGODB_URI	No	MongoDB Atlas connection string (omit for SQLite)
PORT	No	Server port (default: 4000)
CLIENT_URL	Yes	CORS origin (e.g. http://localhost:5173)
NODE_ENV	No	development production

13. Local Setup Guide

1. Clone the repository

```
git clone https://github.com//readme-auto-generator.git
cd readme-auto-generator
```

2. Install dependencies

```
npm install          # installs root + client + server workspaces
```

3. Configure environment

```
cp .env.example .env

# Edit .env and add ANTHROPIC_API_KEY, GITHUB_CLIENT_ID, etc.
```

4. Start development servers

```
npm run dev          # starts Vite (port 5173) + Express (port 4000) concurrently
```

5. Open the app

http://localhost:5173

6. Run tests

npm test # runs Jest (server) + Vitest (client)

SUMMARY

14. Conclusion

README Auto Generator demonstrates the practical application of large language models as a developer-productivity tool. By combining careful static analysis of code artefacts with a well-structured prompt, the system produces documentation that is meaningfully better than any generic template — without requiring the developer to write a single line of Markdown.

The architecture is intentionally simple, observable, and extensible. Future directions include: multi-language i18n for generated docs, semantic code search integration (e.g. code embeddings for richer context), VS Code extension, and a diff-aware update mode that edits only the sections that changed between commits.

Aspect	Choice / Value
Primary language	JavaScript / TypeScript (Node.js 20 + React 18)
AI model	claude-sonnet-4-20250514 (Anthropic)
Database	SQLite (dev) · MongoDB Atlas Free (prod)
Auth	GitHub OAuth 2.0 + JWT (HttpOnly cookie)
Hosting (optional)	Vercel (frontend) + Render (backend) — both free
License	MIT